Objective

Design and implement a **low-latency in-memory Limit Order Book** in modern C++ (C++17 or later).
The goal is to simulate how exchanges maintain real-time bid/ask data with microsecond-level performance.

You are expected to design a data structure that can:

- Maintain bid and ask sides efficiently
- Insert, cancel, and amend orders
- Provide aggregated snapshots to a given depth
- Be cache-friendly and minimize heap allocations. (Bonus if you can use a memory Pool)

---

2. System Requirements

Core operations supported

1. Add Order
2. Cancel Order
3. Amend Order
4. Get Order Book Snapshot (Top N levels)
5. Print Book

---

3. Data Model

Each order must have:

```
struct Order {
    uint64_t order_id;     // Unique order identifier
    bool is_buy;           // true = buy, false = sell
    double price;          // Limit price
    uint64_t quantity;     // Remaining quantity
    uint64_t timestamp_ns; // Order entry timestamp in nanoseconds
};
```

---

4. Core Class Interfaces

Students must implement the following interface in `order_book.cpp`:

```
#pragma once
#include <cstdint>
#include <vector>
#include <string>

struct Order;
struct PriceLevel;

class OrderBook {
public:
    // Insert a new order into the book
    void add_order(const Order& order);

    // Cancel an existing order by its ID
    bool cancel_order(uint64_t order_id);

    // Amend an existing order's price or quantity
    bool amend_order(uint64_t order_id, double new_price, uint64_t
new_quantity);

    // Get a snapshot of top N bid and ask levels (aggregated
quantities)
    void get_snapshot(size_t depth, std::vector<PriceLevel>& bids,
std::vector<PriceLevel>& asks) const;

    // Print current state of the order book
    void print_book(size_t depth = 10) const;
};
```

Supporting Types

PriceLevel

Represents a price level with aggregated volume.

```
struct PriceLevel {
    double price;
    uint64_t total_quantity;
};
```

Order Lookup Structure

Students should maintain a lookup table for O(1) cancel/amend:

```
std::unordered_map<uint64_t, Order*> order_lookup;
```

---

5. Behavior Rules

Add Order

- Insert the order into the appropriate side (bids or asks).
- Within the same price level, maintain FIFO (first-in-first-out) priority.
- Multiple orders can exist at the same price.
- If a price level already exists, aggregate the total quantity.

Cancel Order

- Remove the order by `order_id`
- Update the total quantity at that price level.
- Remove the price level if it becomes empty.

Amend Order

- If price changes → treat it as cancel + add.
- If only quantity changes → update in place.
- Return `false` if `order_id` is not found.

Snapshot

- Return top N bids (highest prices) and asks (lowest prices).
- Aggregate quantities per price level.

Matching (Optional)

You are not required to implement matching.
For extra credit, implement basic matching when `best_bid >= best_ask`.

Deadline
19th October 2025

Write your submissions here in PR to this repo
https://github.com/KnightKnight27/Scaler-HFT-2027